TechCorp Engineering Handbook

Development Standards & Best Practices

Version 2.0 | Last Updated: February 2026

Table of Contents

---

1. Introduction

Welcome to the TechCorp Engineering Team! This handbook serves as the definitive guide to our development practices, workflows, and technical standards. Whether you're a new engineer or a seasoned team member, this document will help you understand how we build, deploy, and maintain software at TechCorp.

Our Engineering Principles

Quality Over Speed: We prioritize writing maintainable, well-tested code over rushing features to production.

Collaboration First: Code reviews, pair programming, and knowledge sharing are core to our culture.

Automation Everything: If you do it twice, automate it. Our CI/CD pipeline handles the repetitive work.

Fail Fast, Learn Faster: We encourage experimentation, own our mistakes, and continuously improve.

Security by Default: Security is not an afterthought; it's baked into every stage of development.

---

2. Git Workflow & Version Control

Branch Strategy

We follow a modified Git Flow workflow optimized for continuous deployment.

Branch Types

main (protected)

- Production-ready code only
- Always deployable
- Requires pull request + 2 approvals
- Auto-deploys to production on merge
- Protected from force pushes and deletions

develop (protected)

- Integration branch for features
- Reflects latest delivered development changes
- Auto-deploys to staging environment
- Requires pull request + 1 approval

feature/*

- New features and non-emergency fixes
- Branch from: develop
- Merge back to: develop
- Naming: feature/JIRA-123-short-description
- Example: feature/ENG-456-user-authentication

hotfix/*

- Emergency production fixes

- Branch from: main
- Merge to: main AND develop
- Naming: hotfix/JIRA-123-critical-bug
- Example: hotfix/ENG-789-payment-gateway-fix

release/*

- Preparing production releases
- Branch from: develop
- Merge to: main AND develop
- Naming: release/v1.2.3
- Only bug fixes, documentation, and release prep allowed

Commit Standards

Commit Message Format

<type>(<scope>): <subject>

<body>

<footer>

Types:

- **feat:** New feature
- **fix:** Bug fix
- **docs:** Documentation only changes
- **style:** Code formatting (no functional changes)
- **refactor:** Code restructuring without changing behavior
- **test:** Adding or updating tests
- **chore:** Maintenance tasks, dependencies

Example:

feat(auth): implement OAuth2 login flow

- Add Google OAuth provider
- Create user session management
- Update login UI components

Closes ENG-456

Commit Best Practices

- Keep commits atomic and focused on a single change
- Write descriptive commit messages (50 chars for subject, 72 for body)
- Link to JIRA ticket in commit message
- Sign commits with GPG key for security
- Never commit secrets, API keys, or credentials
- Use .gitignore to exclude build artifacts and local configs

Working with Branches

Creating a Feature Branch

```
# Update your local develop branch
git checkout develop
git pull origin develop

# Create and checkout feature branch
git checkout -b feature/ENG-123-new-feature

# Push to remote and set upstream
git push -u origin feature/ENG-123-new-feature
```

Keeping Your Branch Updated

Rebase on develop regularly (at least daily)

git checkout develop

git pull origin develop

git checkout feature/ENG-123-new-feature

git rebase develop

Resolve conflicts if any

Force push after rebase (only to your feature branch!)

git push --force-with-lease

Merging vs Rebasing

- **Rebase:** Use for updating feature branches with latest develop
- **Merge:** Use for integrating feature branches into develop/main
- Never rebase shared branches (develop, main)
- Always use --force-with-lease instead of --force

Pull Request Requirements

Before creating a pull request, ensure:

✓ Branch is up-to-date with target branch ✓ All tests pass locally ✓ Code follows style guidelines (linter passes) ✓ No debug statements or commented code ✓ Updated relevant documentation ✓ Added/updated tests for new functionality ✓ Checked for security vulnerabilities ✓ JIRA ticket linked in description

---

3. Code Review Process

Code review is mandatory for all changes to develop and main branches. It's a learning opportunity and quality gate, not a criticism session.

Review Timeline

- **First Review:** Within 4 business hours of PR creation
- **Response to Feedback:** Within 24 hours

- **Final Approval:** Within 48 hours of initial submission
- **Urgent/Hotfix:** Within 1 hour (notify in #eng-urgent Slack)

Reviewer Assignment

Automatic Assignment

Our GitHub bot automatically assigns reviewers based on:

- Code ownership (CODEOWNERS file)
- Team membership and expertise
- Current review workload

Manual Assignment

Request specific reviewers when:

- Domain expertise needed
- Architectural changes require senior review
- Security-sensitive code needs security team review

Review Guidelines

For Authors

Creating the Pull Request:

- Use PR template (auto-populated)
- Provide clear description of changes
- Link JIRA ticket and related PRs
- Add screenshots/videos for UI changes
- Self-review your own code first
- Keep PRs small (<400 lines changed when possible)

Responding to Feedback:

- Address all comments (resolve or discuss)
- Push fixes as new commits (don't amend during review)
- Re-request review after addressing feedback

- Thank reviewers for their time and insights
- Don't take criticism personally; focus on code quality

For Reviewers

What to Look For:

Functionality:

- Does the code do what it's supposed to?
- Are edge cases handled?
- Is error handling appropriate?

Code Quality:

- Is the code readable and maintainable?
- Are naming conventions followed?
- Is there unnecessary complexity?
- Are there code smells or anti-patterns?

Testing:

- Are there sufficient unit tests?
- Do tests cover happy path and edge cases?
- Are integration tests needed?

Security:

- Are inputs validated and sanitized?
- Are authentication/authorization checks present?
- Are secrets properly managed?
- Are there SQL injection or XSS vulnerabilities?

Performance:

- Are there obvious performance issues?
- Are database queries optimized?
- Is caching used appropriately?

Documentation:

- Are complex functions documented?
- Is README updated if needed?
- Are API changes reflected in documentation?

Review Categories

Approve: Code is ready to merge **Request Changes:** Issues must be fixed before merge **Comment:** Suggestions or questions, but not blocking

Review Checklist

- [ ] Code functionality verified

- [ ] Tests are comprehensive

- [ ] No security vulnerabilities

- [ ] Follows style guidelines

- [ ] Documentation updated

- [ ] No performance concerns

- [ ] Error handling is appropriate

- [ ] Breaking changes documented

Handling Disagreements

1. **Discuss:** Use PR comments to explain your perspective
2. **Sync Meeting:** Schedule 15-min call if async discussion stalls
3. **Escalate:** Involve tech lead if consensus can't be reached
4. **Timebox:** Don't let perfect be enemy of good; iterate later if needed

---

4. Deployment Procedures

Deployment Pipeline

Our automated CI/CD pipeline handles most deployments, but understanding the process is crucial.

Environments

Development (dev.techcorp.com)

- Auto-deploys from feature branches on push
- Isolated environment per feature branch
- Temporary (deleted after PR merge)
- Uses development AWS account

Staging (staging.techcorp.com)

- Auto-deploys from develop branch
- Production-like environment
- Used for QA and integration testing
- Mirrors production infrastructure

Production (techcorp.com)

- Deploys from main branch
- Requires manual approval for deployment
- Blue-green deployment strategy
- Automatic rollback on failure

Deployment Process

For Regular Features:

1. **Merge to develop** → Auto-deploys to staging
2. **QA Testing** → Verify on staging environment
3. **Create Release PR** → main ← develop
4. **Get Approvals** → 2 senior engineers + tech lead
5. **Merge to main** → Triggers production deployment pipeline
6. **Manual Approval** → On-call engineer approves in CircleCI
7. **Deploy** → Blue-green deployment executes
8. **Monitor** → Watch logs and metrics for 30 minutes
9. **Announce** → Post in #deployments Slack channel

For Hotfixes:

1. **Create hotfix branch** from main
2. **Fix and test** locally
3. **Create PR** → main ← hotfix/ENG-XXX
4. **Get urgent review** → Tag on-call engineer
5. **Merge and deploy** → Follow same deployment steps
6. **Backport to develop** → Ensure fix is in develop branch

Pre-Deployment Checklist

Before approving a production deployment:

✓ All staging tests passing ✓ QA sign-off received ✓ Database migrations tested ✓ Feature flags configured ✓ Rollback plan documented ✓ Monitoring alerts configured ✓ On-call engineer identified ✓ Stakeholders notified of deployment

Deployment Commands

Deploy to staging (automatic)

git push origin develop

Deploy to production (requires approval)

git push origin main

Manual deployment (emergency only)

./scripts/deploy.sh --env production --version v1.2.3

Rollback to previous version

./scripts/rollback.sh --env production --version v1.2.2

Database Migrations

Migration Guidelines:

- Always write backward-compatible migrations

- Test migrations on staging with production data copy
- Use transactions for data migrations
- Plan for rollback scenarios
- Never delete columns immediately (deprecate first)

Migration Process:

```
# migrations/20260204_add_user_email_verified.py


def upgrade():
  # Add column with default value
  op.add_column('users',
    sa.Column('email_verified', sa.Boolean(),
    nullable=False, server_default='false'))


def downgrade():
  # Rollback changes
  op.drop_column('users', 'email_verified')
```

Rollback Procedures

Automatic Rollback:

- Triggered if health checks fail post-deployment
- Reverts to previous version automatically
- Notifications sent to #eng-alerts

Manual Rollback:

```
# Immediate rollback
./scripts/rollback.sh --env production --immediate


# Verify rollback
```

curl https://techcorp.com/health

Post-Deployment

Monitoring (first 30 minutes):

- Watch error rates in Datadog
- Check application logs in CloudWatch
- Monitor API response times
- Verify database performance
- Check user-reported issues in Sentry

Deployment Announcement Template:

🚀 Production Deployment: v1.2.3

Changes:

- Feature: User email verification (ENG-456)

- Fix: Payment gateway timeout (ENG-789)

Deployed by: @john

Monitoring: @oncall-engineer

Dashboard: https://datadog.techcorp.com/deploy-123

---

5. On-Call Rotation Policy

On-Call Schedule

Primary On-Call: 24/7 responsibility, one-week rotations **Secondary On-Call:** Backup support, escalation point **Rotation:** Monday 9 AM to following Monday 9 AM (EST)

Rotation Schedule

Engineers rotate through on-call in the following order:

- Week 1: Engineer A (primary), Engineer B (secondary)
- Week 2: Engineer B (primary), Engineer C (secondary)
- Week 3: Engineer C (primary), Engineer D (secondary)
- Continues rotating through full team

Schedule published in PagerDuty and #eng-oncall Slack channel.

On-Call Responsibilities

Primary On-Call Engineer

Incident Response:

- Acknowledge alerts within 5 minutes
- Begin investigation within 10 minutes
- Provide status updates every 30 minutes during incidents
- Engage secondary on-call if needed
- Document all incidents in post-mortem template

Deployment Approval:

- Review and approve production deployments
- Monitor deployments for first 30 minutes
- Execute rollbacks if necessary

Support:

- Monitor #eng-support for critical customer issues
- Triage and escalate bugs to appropriate team
- Support customer success team with technical questions

Secondary On-Call Engineer

- Available for escalations from primary
- Cover primary during brief unavailability
- Assist with complex or critical incidents
- Take over if primary becomes unavailable

Incident Severity Levels

SEV 1 - Critical

- Complete service outage
- Data loss or security breach
- Response time: Immediate
- Update frequency: Every 15 minutes

SEV 2 - Major

- Significant feature degradation
- Elevated error rates (>5%)
- Response time: 15 minutes
- Update frequency: Every 30 minutes

SEV 3 - Minor

- Limited impact to users
- Non-critical bugs
- Response time: 1 hour
- Update frequency: As needed

SEV 4 - Monitoring

- Warning thresholds exceeded
- Potential future issues
- Response time: Next business day

Incident Response Process

Step 1: Acknowledge and Assess (0-5 minutes)

- Acknowledge alert in PagerDuty
- Check Datadog dashboard for scope
- Determine severity level
- Create incident channel in Slack (#incident-YYYYMMDD-description)

Step 2: Communicate (5-10 minutes)

- Post initial status in #incidents
- Notify stakeholders if SEV 1 or SEV 2
- Update status page if customer-facing impact

Step 3: Investigate and Mitigate (10+ minutes)

- Review recent deployments and changes
- Check application and infrastructure logs
- Identify root cause
- Implement fix or rollback
- Engage secondary on-call or specialists if needed

Step 4: Resolve and Monitor

- Verify fix resolves issue
- Monitor metrics for 30 minutes
- Update status page
- Mark incident as resolved

Step 5: Post-Mortem (within 48 hours)

- Document timeline of events
- Identify root cause
- List action items to prevent recurrence
- Share learnings with team

On-Call Best Practices

Preparation:

- Test PagerDuty notifications before your rotation
- Ensure you have VPN access and admin credentials
- Familiarize yourself with runbooks
- Review recent incidents and alerts

During Rotation:

- Keep laptop charged and nearby
- Stay within cell reception when possible
- Limit alcohol consumption
- Notify team if you need to hand off temporarily
- Don't hesitate to escalate; it's better to be safe

Communication:

- Use #incident channels for all communication
- Keep updates concise and actionable
- Avoid jargon when notifying non-technical stakeholders
- Document everything; memory is unreliable during incidents

On-Call Compensation

- **Stipend:** $500 per week of primary on-call
- **Incident Pay:** $100 per hour for off-hours incident work (SEV 1-2)
- **Time Off:** Comp day if weekend incident exceeds 4 hours
- **Schedule Swap:** Coordinate swaps via #eng-oncall (manager approval needed)

Escalation Paths

Technical Escalation:

1. Secondary on-call engineer
2. Team tech lead
3. Engineering manager
4. VP of Engineering

Business Escalation:

1. Customer success manager (customer impact)
2. Product manager (feature/product questions)
3. VP of Product (major product decisions)

---

6. Tech Stack Documentation

Backend: Python

Framework and Libraries

Django 4.2

- Web framework for API and admin interface
- Django REST Framework for RESTful APIs
- Celery for asynchronous task processing
- Django ORM for database interactions

Key Libraries:

- **requests:** HTTP client for external API calls
- **boto3:** AWS SDK for S3, SQS, DynamoDB interactions
- **pytest:** Testing framework
- **black:** Code formatter
- **flake8:** Linter for style enforcement
- **mypy:** Static type checker
- **sentry-sdk:** Error tracking and monitoring

Python Standards

Version: Python 3.11+

Code Style:

- Follow PEP 8 guidelines
- Use Black formatter (line length: 100)
- Use type hints for all function signatures
- Document complex functions with docstrings

Example:

```
from typing import List, Optional
```

```
from datetime import datetime


def calculate_user_metrics(
    user_id: int,
    start_date: datetime,
    end_date: Optional[datetime] = None
) -> List[dict]:
    """
    Calculate user engagement metrics for a given period.

    Args:
        user_id: ID of the user
        start_date: Start of calculation period
        end_date: End of period (defaults to now)

    Returns:
        List of metric dictionaries with keys:
        - metric_name, value, timestamp

    Raises:
        UserNotFoundError: If user_id doesn't exist
    """
    # Implementation
    pass
```

Project Structure

```
backend/
```

```
├── apps/
│   ├── users/        # User management
│   ├── auth/         # Authentication
│   ├── payments/     # Payment processing
│   └── analytics/    # Analytics engine
├── config/           # Django settings
├── tests/            # Test suite
├── scripts/          # Utility scripts
├── requirements/     # Dependencies
│   ├── base.txt
│   ├── dev.txt
│   └── prod.txt
└── manage.py
```

Testing

Test Coverage Target: >80%

Test Types:

- Unit tests: Individual function testing
- Integration tests: API endpoint testing
- E2E tests: Critical user flows

Running Tests:

```
# All tests
pytest


# Specific module
pytest apps/users/tests/
```

```
# With coverage report
pytest --cov=apps --cov-report=html


# Fast tests only (skip slow integration tests)
pytest -m "not slow"
```

Frontend: React

Framework and Libraries

React 18.2

- Create React App for bootstrapping
- React Router for navigation
- Redux Toolkit for state management
- React Query for server state

Key Libraries:

- **axios:** HTTP client
- **formik:** Form handling
- **yup:** Schema validation
- **tailwindcss:** Utility-first CSS framework
- **jest:** Testing framework
- **react-testing-library:** Component testing

React Standards

Component Structure:

```
// Use functional components with hooks
import { useState, useEffect } from 'react';


const UserProfile = ({ userId }) => {
```

```
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchUser(userId).then(setUser).finally(() => setLoading(false));
  }, [userId]);

  if (loading) return <Spinner />;
  if (!user) return <ErrorMessage />;

  return (
    <div className="user-profile">
      <h1>{user.name}</h1>
      <p>{user.email}</p>
    </div>
  );
};

export default UserProfile;
```

Best Practices:

- Use functional components over class components
- Keep components small and focused (<200 lines)
- Extract reusable logic into custom hooks
- Use TypeScript for type safety (migrating from JavaScript)
- Implement error boundaries for graceful failures
- Memoize expensive computations with useMemo

- Use React.memo for expensive render optimizations

Project Structure

```
frontend/
├── src/
│   ├── components/    # Reusable UI components
│   ├── pages/         # Page components
│   ├── hooks/         # Custom hooks
│   ├── services/      # API clients
│   ├── store/         # Redux store
│   ├── utils/         # Helper functions
│   └── App.js
├── public/
├── tests/
└── package.json
```

Testing

Test Coverage Target: >75%

```
# Run tests
npm test


# With coverage
npm test -- --coverage


# E2E tests (Cypress)
npm run cypress:open
```

Infrastructure: AWS

Core Services

Compute:

- **ECS (Elastic Container Service):** Container orchestration
- **Fargate:** Serverless container compute
- **Lambda:** Serverless functions for event processing

Storage:

- **S3:** Object storage for user uploads and static files
- **RDS PostgreSQL:** Primary relational database
- **ElastiCache Redis:** Caching and session storage
- **DynamoDB:** NoSQL for high-throughput use cases

Networking:

- **VPC:** Network isolation
- **ALB:** Application load balancing
- **Route 53:** DNS management
- **CloudFront:** CDN for static assets

Monitoring & Logging:

- **CloudWatch:** Logs and basic metrics
- **Datadog:** Advanced monitoring and APM
- **Sentry:** Error tracking
- **PagerDuty:** On-call alerting

CI/CD:

- **CircleCI:** Build and test automation
- **AWS CodeDeploy:** Deployment orchestration
- **Docker:** Container images

Infrastructure as Code

We use Terraform for infrastructure management:

```
# terraform/production/main.tf

module "app_cluster" {
  source = "../../modules/ecs-cluster"

  cluster_name    = "production-app"
  instance_type   = "t3.large"
  desired_capacity = 5
  max_capacity    = 20

  vpc_id     = module.vpc.id
  subnet_ids = module.vpc.private_subnet_ids
}
```

Terraform Commands:

```
# Plan changes
terraform plan -out=tfplan

# Apply changes (requires approval)
terraform apply tfplan

# Never apply directly to production without review
```

AWS Best Practices

- Use IAM roles, never hardcode credentials
- Enable MFA for all AWS console access
- Tag all resources for cost tracking
- Use multiple availability zones for HA

- Enable encryption at rest and in transit
- Regular backup and disaster recovery testing
- Monitor costs with AWS Cost Explorer

---

7. Best Practices & Standards

Security

Authentication & Authorization:

- Use OAuth 2.0 for third-party integrations
- Implement JWT tokens with short expiration (15 min access, 7 day refresh)
- Enforce role-based access control (RBAC)
- Never store passwords in plain text (use bcrypt with salt)

Data Protection:

- Encrypt sensitive data at rest (AES-256)
- Use TLS 1.3 for all communications
- Sanitize all user inputs to prevent XSS
- Use parameterized queries to prevent SQL injection
- Implement rate limiting on all APIs (100 requests/min per user)

Secrets Management:

- Store secrets in AWS Secrets Manager
- Rotate secrets every 90 days
- Never commit secrets to Git
- Use environment variables for configuration

Performance

Database:

- Add indexes for frequently queried fields
- Use connection pooling (max 20 connections per instance)

- Implement query result caching for expensive operations
- Monitor slow queries (>500ms) and optimize

API:

- Implement pagination for list endpoints (max 100 items)
- Use compression for responses >1KB
- Cache frequently accessed data in Redis (TTL 5-60 min)
- Return 304 Not Modified for unchanged resources

Frontend:

- Code splitting for route-based bundles
- Lazy load images and heavy components
- Minimize bundle size (target <200KB gzipped)
- Use CDN for static assets

Documentation

Code Documentation:

- Comment complex logic and business rules
- Keep comments updated with code changes
- Use docstrings for public APIs
- Avoid obvious comments

API Documentation:

- Maintain OpenAPI (Swagger) specification
- Document all endpoints, parameters, responses
- Provide example requests and responses
- Keep docs in sync with implementation

README Files:

- Setup instructions for local development
- Architecture overview

- Deployment procedures
- Troubleshooting guide

---

8. Resources & Support

Useful Links

- **Code Repository:** github.com/techcorp/platform
- **Documentation:** docs.techcorp.com
- **Monitoring:** datadog.techcorp.com
- **Error Tracking:** sentry.io/techcorp
- **CI/CD:** circleci.com/techcorp
- **API Docs:** api.techcorp.com/docs

Slack Channels

- **#engineering:** General engineering discussion
- **#eng-backend:** Backend-specific topics
- **#eng-frontend:** Frontend-specific topics
- **#eng-oncall:** On-call coordination
- **#incidents:** Active incident response
- **#deployments:** Deployment announcements
- **#eng-support:** Customer support escalations

Getting Help

Technical Questions:

1. Search internal documentation
2. Ask in relevant Slack channel
3. Schedule office hours with tech leads (Tuesdays/Thursdays 2-3 PM)

Emergency Support:

- Page on-call engineer via PagerDuty

- Post in #eng-urgent for critical issues
- Call engineering manager for business-critical emergencies

Continuous Learning

Weekly:

- Tech talks every Friday at 3 PM
- Code review feedback sessions

Monthly:

- Engineering all-hands
- Incident post-mortem reviews

Quarterly:

- Engineering retreat
- Training budget: $1,000 per engineer

Welcome to the team! Questions? Reach out in #engineering or ping your onboarding buddy.

This handbook is a living document. Submit updates via PR to docs/engineering-handbook.md